Ejercicio práctico 1.3

«Solicitud, respuesta y JSON en el taller de bicicletas»

Nombre	Anthony Rodríguez
Curso	3ro Matutino, Desarrollo de Software
Fecha	24 de octubre del 2026

Asignatura	Herramientas informáticas para el despliegue de diagramas
Unidad	1. Principios de la arquitectura web en el diseño y desarrollo de aplicaciones backend
Tema	1.3 Solicitud y respuesta HTTP. Formato JSON
Docente	Jhostin Omar Vacacela Saca
Duración	90 minutos en clase (partes A a C) y 1 hora de trabajo autónomo (partes D y E)
Modalidad	Individual

Objetivo

Crear un diagrama de una solicitud y una respuesta HTTP con datos en JSON, e implementar con Kotlin y Spring Boot un servicio cuya respuesta se analiza.

Situación

En el taller de bicicletas «La Cadena» las reparaciones se anotan en un cuaderno, y los clientes llaman por teléfono para preguntar si su bicicleta ya está lista. El dueño quiere un servicio web que pueda consultar la página del taller y una aplicación móvil. Cada reparación guarda la bicicleta, los datos del cliente, la lista de repuestos, el costo y si ya está lista.

Primero vas a describir cómo viajan los datos entre el cliente y el servidor. Después vas a construir el servicio.

Instrucciones generales

- Las partes A, B y C se resuelven en clase, a mano o en el computador. Las partes D y E son el trabajo autónomo.
- Entrega un archivo .zip con este documento respondido, la imagen o foto del diagrama de la parte B y el proyecto de la parte D sin la carpeta target.
- Usa JDK 21 y el wrapper de Maven (mvnw). Los pasos son los de la guía «Kotlin y Spring Boot 3 con Maven».
- Se permite usar IA si declaras la herramienta y el alcance en la parte E. Como referencia, el reporte de Turnitin no debe pasar de 30 % de contenido posiblemente generado por IA ni de 10 % de similitud.
- El puntaje de cada parte está a la vista; el total es de 100 puntos.

Parte A. Leer y predecir (20 puntos)

Esta es una solicitud real enviada al servicio del taller, y la respuesta que devolvió. Léelas con calma antes de responder.

Solicitud

```
POST /reparaciones HTTP/1.1
Host: localhost:8080
Content-Type: application/json
Accept: application/json
Content-Length: 124

{"bicicleta":"Bicicleta urbana","cliente":{"nombre":"Rosa
Vera","telefono":"0961112233"},"repuestos":["timbre"],"costo":6.5}
```

Respuesta

```
HTTP/1.1 201
Location: /reparaciones/3
Content-Type: application/json

{"id":3,"bicicleta":"Bicicleta urbana","cliente":{"nombre":"Rosa
Vera","telefono":"0961112233"},"repuestos":["timbre"],"costo":6.5,"lista":false}
```

A1 (4 puntos). Escribe el método, la ruta y la versión del protocolo de la solicitud.

Respuesta: Método: POST; Ruta: /reparaciones; Versión: HTTP/1.1

A2 (4 puntos). Nombra las cabeceras de la solicitud y explica qué le indica al servidor la cabecera Content-Type: application/json.

Respuesta: Las cabeceras son Host, Content-Type, Accept y Content-Length. Sirven para que el servidor sepa como leer el body. La cabecera Content-Type: application/json indica que el body tiene el formato JSON.

A3 (4 puntos). ¿Qué código de estado devolvió el servicio y qué significa? ¿Para qué sirve la cabecera Location?

Respuesta: Devolvió el código 201 que significa Created. La cabecera Location es la ruta del nuevo recurso creado, para que el cliente sepa en donde encontrarlo.

A4 (4 puntos). Completa el tipo JSON de cada campo del cuerpo de la solicitud (string, number, boolean, array u object).

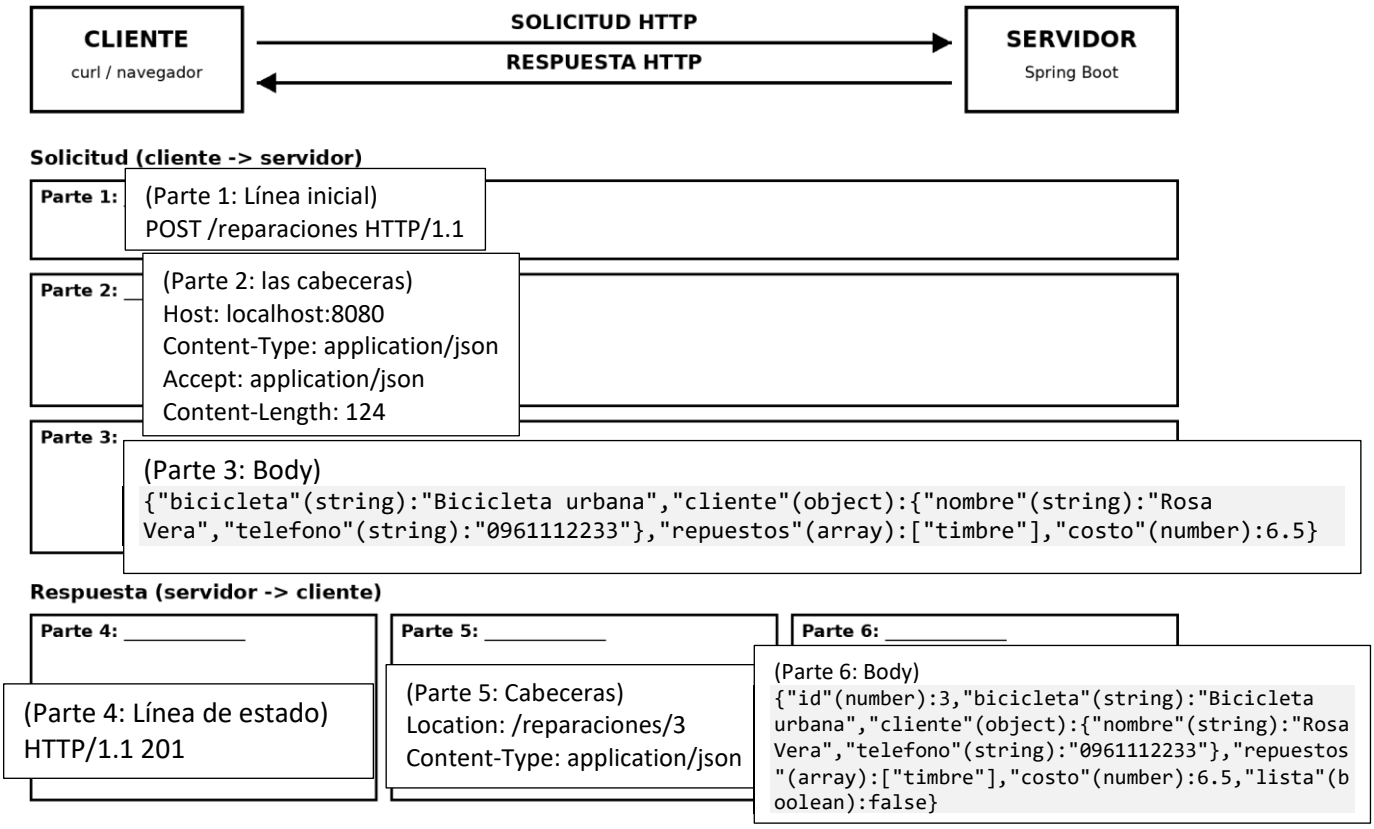
Campo	**Tipo JSON**
bicicleta	string
cliente	object
repuestos	array
costo	number

A5 (4 puntos). Predice: si la misma solicitud se enviara con Content-Type: text/plain en lugar de application/json, ¿qué código de estado devolvería el servicio? Justifica tu respuesta. Luego podrás comprobarlo en la parte D.

Respuesta: En este caso devolvería el código 415 que es de Unsupported Media Type, ya que el servidor espera el formato JSON.

Parte B. Diagrama de solicitud y respuesta (20 puntos)

Con los datos de la parte A, completa el diagrama: nombra cada una de las seis partes (por ejemplo «Parte 1: Línea inicial») y escribe en ella el contenido real. Puedes dibujarlo a mano sobre esta hoja o rehacerlo en draw.io; en ese caso, adjunta la imagen en el .zip.



Presenta tu diagrama en plenaria durante dos minutos: explica qué hace cada parte y en qué parte viaja el JSON.

Parte C. Detectar y corregir errores (20 puntos)

El ayudante del taller escribió este JSON para registrar una reparación, pero tiene cuatro errores de sintaxis o de valor. Cada error está en una línea distinta.

```
{
  'bicicleta': "Bicicleta plegable",
  "cliente": {"nombre": "Ana Mora", "telefono": 0987654321},
  "repuestos": ["cadena", "luz"],
  "costo": 12,50,
  "lista": Falso
}
```

C1 (12 puntos). Completa la tabla: la línea del error, qué está mal y cómo se corrige.

Línea	Qué está mal	Corrección
2	Puso comillas simples. Pero JSON solo admite comillas dobles	"bicicleta":"Bicicleta plegable"
3	El valor de "telefono" está sin comillas. Ese es el formato de number, así que sería inválido y daría un error de sintaxis.	"telefono":"0987654321"
5	El valor de "costo" tiene coma para separar decimales. Hay que usar punto.	"costo":12.50
6	El valor de "lista", que es un boolean, solo tiene dos valores: false y true. Y en minúsculas. No "Falso" en español	"lista":false

C2 (4 puntos). Escribe el JSON completo y corregido.

Respuesta:

```
{
  "bicicleta": "Bicicleta plegable",
  "cliente": {"nombre":"Ana Mora", "telefono":"0987654321"},
  "repuestos": ["cadena","luz"],
  "costo": 12.50,
  "lista": false
}
```

C3 (4 puntos). Si el JSON original se enviara por POST al servicio de la parte D, ¿qué código de estado devolvería? Explica por qué.

Respuesta: Lo más seguro es que devuelva el código 400 Bad Request, debido a los errores de sintaxis; por lo que no se convierte en objeto.

Parte D. Implementar el servicio (30 puntos)

Construye un servicio con Kotlin, Spring Boot 3 y Maven para el recurso /reparaciones. Sigue los pasos de la guía; aquí solo se indica qué debe quedar.

D1. El proyecto Maven (6 puntos)

Genera el proyecto en start.spring.io con Project = Maven, Language = Kotlin, Group = ec.edu, Artifact = taller-api, Packaging = Jar, Java = 21 y la dependencia Spring Web. Como start.spring.io ofrece Spring Boot 4.x, ajusta el pom.xml para que use Spring Boot 3.5.16 y Kotlin 2.0.21, con las dependencias de la guía.

Pega aquí las líneas del bloque <parent> y de <properties> de tu pom.xml.

```
Tu pom.xml: <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.16</version>
    <relativePath/>
</parent>

<properties>
    <java.version>21</java.version>
    <kotlin.version>2.0.21</kotlin.version>
</properties>
```

D2. El modelo (8 puntos)

Escribe tres data class con estos campos:

Clase	Campos
Cliente	nombre (String), telefono (String)
Reparacion	id (Int), bicicleta (String), cliente (Cliente), repuestos (lista de String), costo (Double), lista (Boolean)
NuevaReparacion	bicicleta (String), cliente (Cliente), repuestos (lista de String, vacía por defecto), costo (Double). No lleva id ni lista: los asigna el servidor.

Pega aquí el código de las tres clases.

Tu código:

```
package ec.edu.taller_api

data class Cliente(
    val nombre: String,
    val telefono: String,
)

data class Reparacion(
    val id: Int,
    val bicicleta: String,
    val cliente: Cliente,
    val repuestos: List<String>,
    val costo: Double,
    val lista: Boolean,
)

data class NuevaReparacion(
    val bicicleta: String,
    val cliente: Cliente,
    val repuestos: List<String> = emptyList(),
    val costo: Double,
)
```

D3. El controlador (10 puntos)

Escribe un controlador con el prefijo /reparaciones y estas tres operaciones. Empieza con dos reparaciones guardadas en una lista en memoria.

Operación	Debe hacer
GET /reparaciones	Devolver la lista completa en JSON
GET /reparaciones/{id}	Devolver la reparación en JSON, o 404 Not Found si el id no existe
POST /reparaciones	Recibir una NuevaReparacion en JSON, asignar el id, dejar lista en false y responder 201 Created con la cabecera Location y la reparación creada en el cuerpo

Pega aquí el código del controlador.

Tu código:

```
@RestController
@RequestMapping("/reparaciones")
class ReparacionesController {

    private val reparaciones = mutableListOf(
        Reparacion(1, "Bicicleta de montaña", Cliente("Luis Paredes",
"0991234567"), listOf("cadena", "frenos"), 15.0, true),
        Reparacion(2, "Bicicleta de ruta", Cliente("María Torres", "0987001122"),
listOf("llanta"), 8.75, false),
    )
    private var siguienteId = 3

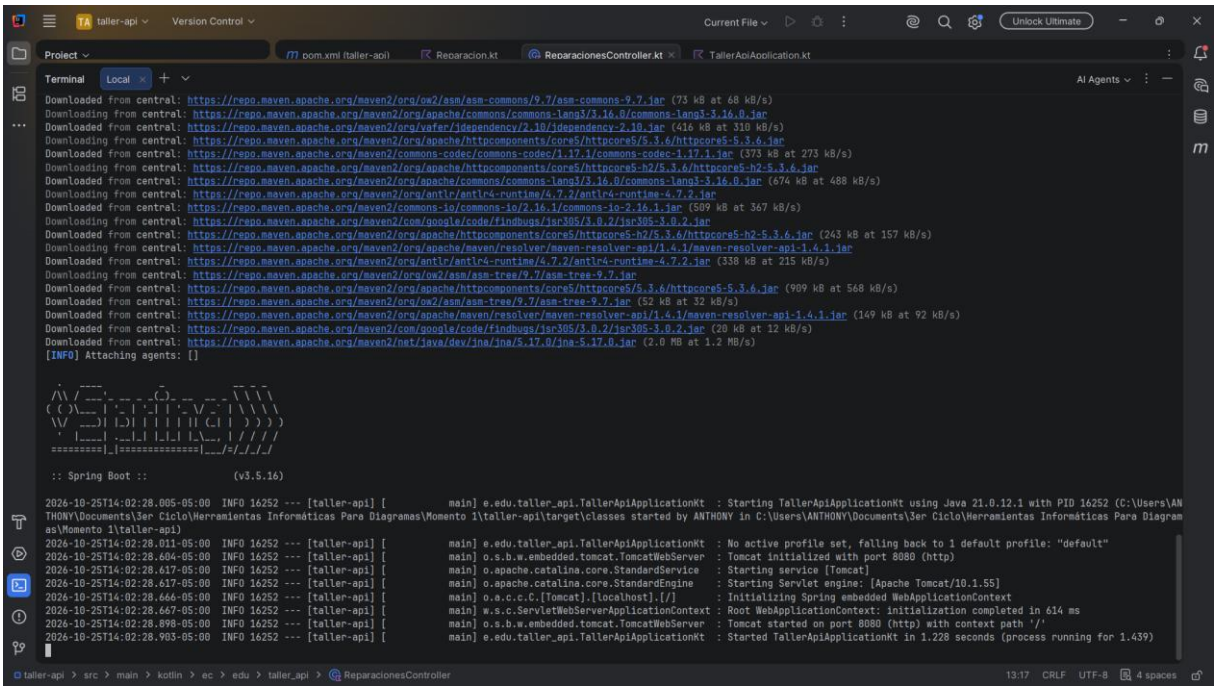
    @GetMapping
    fun listar(): List<Reparacion> = reparaciones

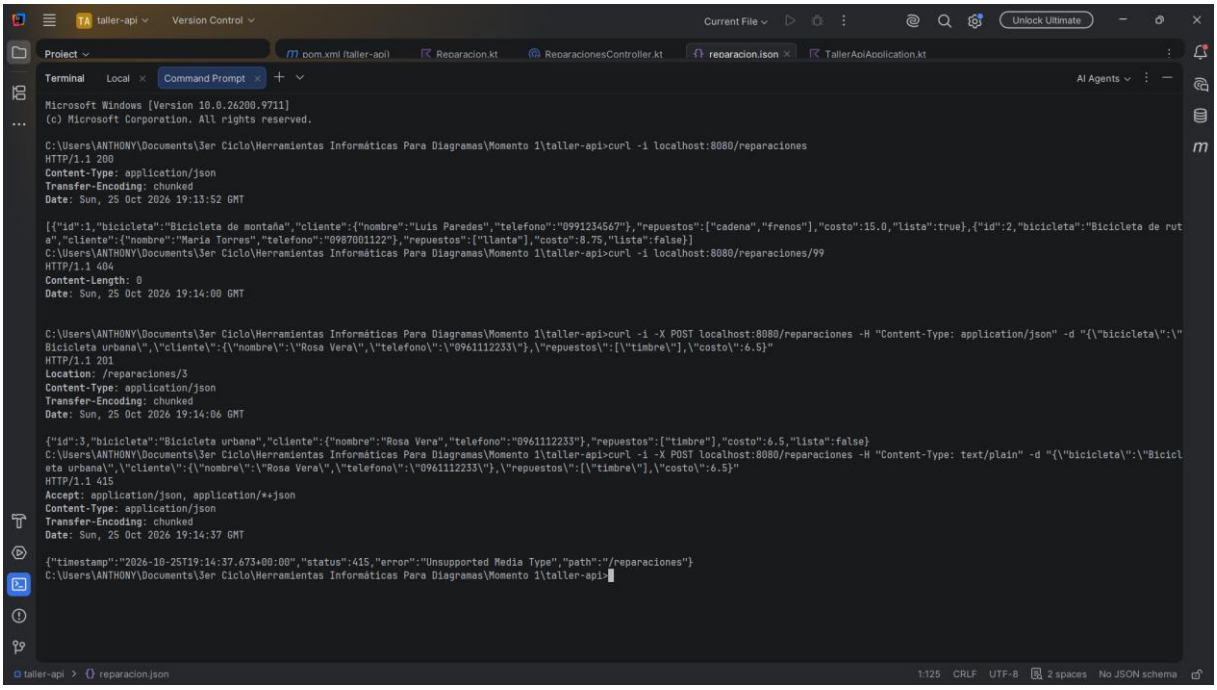
    @GetMapping("/{id}")
    fun obtener(@PathVariable id: Int): ResponseEntity<Reparacion> =
        reparaciones.find { it.id == id }
            ?.let { ResponseEntity.ok(it) }
            ?: ResponseEntity.notFound().build()

    @PostMapping
    fun crear(@RequestBody nueva: NuevaReparacion): ResponseEntity<Reparacion> {
        val reparacion = Reparacion(
            id = siguienteId++,
            bicicleta = nueva.bicicleta,
            cliente = nueva.cliente,
            repuestos = nueva.repuestos,
            costo = nueva.costo,
            lista = false,
        )
        reparaciones.add(reparacion)
        return
ResponseEntity.created(URI("/reparaciones/${reparacion.id}")).body(reparacion)
    }
}
```

D4. Solicitudes y análisis de las respuestas (6 puntos)

Ejecuta el servicio con ./mvnw spring-boot:run (en Windows, mvnw.cmd) y, desde otra terminal, envía estas tres solicitudes con curl -i. En Windows usa curl.exe o Git Bash.





- GET /reparaciones
- GET /reparaciones/99
- POST /reparaciones con el cuerpo JSON de la parte A, la cabecera Content-Type: application/json y la opción -X POST

Completa la tabla con lo que realmente devolvió tu servicio.

Solicitud	Código	Content-Type de la respuesta	Cabecera Location	Qué contiene el cuerpo
GET /reparaciones	200	application/json	No hay la cabecera Location	`[{"id":1,"bicicleta":"Bicicleta de montaña","cliente":{"nombre":"Luis Paredes","telefono":"0991234567"},"repuestos":["cadena","frenos"],"costo":15.0,"lista":true},{"id":2,"bicicleta":"Bicicleta de ruta","cliente":{"nombre":"María Torres","telefono":"0987001122"},"repuestos":["llanta"],"costo":8.75,"lista":false}]`
GET /reparaciones/99	404	No hay "Content-Type"	No tiene la cabecera Location	No tiene cuerpo
POST /reparaciones	201	application/json	/reparaciones/3	`{"id":3,"bicicleta":"Bicicleta urbana","cliente":{"nombre":"Rosa Vera","telefono":"0961112233"},"repuestos":["timbre"],"costo":6.5,"lista":false}`

Con el servicio activo, vuelve a la predicción A5: envía el POST con -H "Content-Type: text/plain" y anota el código que devuelve.

Código obtenido y si coincidió con tu predicción:

```
HTTP/1.1 415
Accept: application/json, application/*+json
Content-Type: application/json
Transfer-Encoding: chunked
Date: Sun, 25 Oct 2026 19:14:37 GMT
```

Sí devolvió el código 415; y además indica en el Accept los tipos de cuerpo que sí admite: application/json y application/*+json

Parte E. Reflexión y declaración de uso de IA (10 puntos)

E1 (6 puntos). Explica con tus palabras qué ocurre desde que curl envía el POST hasta que recibes el 201. Menciona la cabecera Content-Type, @RequestBody, la conversión del JSON a un objeto de Kotlin y la cabecera Location. Entre cinco y ocho líneas.

Respuesta: el comando curl junta el método (POST) con la ruta (/reparaciones) y con la versión (HTTP/1.1), con las cabeceras (ej. Content-Type) y el body. Indicar en la cabecera: Content-Type: application/json, le avisa a Spring que el cuerpo de la petición viene en formato JSON. Para traducir lo que dice el body en formato JSON, Spring utiliza Jackson, que convierte ese texto del body en un objeto que puede utilizar Kotlin, para eso sirve la anotación @RequestBody en la función crear(). Después de guardar el objeto, responde con el código 201 Created para confirmar la creación. Al devolver la respuesta, el servidor incluye también la cabecera Location justamente para indicar al cliente cómo puede volver a consultar por dicho objeto.

E2 (4 puntos). Declaración de uso de IA. Si no la usaste, escríbelo.

Herramienta utilizada	ChatGPT
Para qué la usaste	Para consultar conceptos relacionados con JSON, Maven, Spring Boot y Kotlin, para comprender métodos y cabeceras HTTP y para resolver un error de compilación.
Qué verificaste por tu cuenta	Compilar y ejecutar el servicio, y probar las 4 solicitudes curl. Comprobar el código que genera la solicitud de la parte A5.

Registro de trazabilidad de la investigación

Tema: Desarrollo de API REST, formato JSON, protocolo HTTP y fundamentos de Spring Boot, Kotlin y Maven.

N.º	Consulta o prompt utilizado	Objetivo de la consulta	Información o palabras clave obtenidas	Relación con la siguiente consulta
1	En la petición `POST /reparaciones HTTP/1.1`, ¿qué significa cada parte y por qué la ruta pasa a `/reparaciones/3`?	Comprender la estructura de una petición HTTP y la asignación de identificadores.	Método HTTP, ruta, versión HTTP/1.1, `id`, `201 Created`, recurso y colección.	Permitió continuar con la estructura de los datos enviados en las peticiones.
2	¿Cómo identificar y distinguir los tipos JSON string y object? ¿Por qué hay un diccionario dentro de otro diccionario?	Diferenciar los tipos de datos y comprender los objetos anidados.	String, object, array, clave-valor, llaves `{}`, comillas dobles y objetos anidados.	Preparó la comprensión de cómo el servidor interpreta el cuerpo de una petición.
3	¿Qué ocurre cuando la solicitud se envía como `Content-Type: text/plain` en vez de `application/json`?	Comprender la función de la cabecera `Content-Type`.	Tipos MIME, cuerpo HTTP, JSON, texto plano, analizadores de cuerpo y `JSON.parse()`.	Llevó a consultar brevemente las consecuencias y los códigos HTTP posibles.
4	En resumen, ¿qué ocurre y qué código devuelve?	Obtener una explicación breve sobre el uso de `text/plain`.	`400 Bad Request`, `415 Unsupported Media Type`, `200 OK` y `201 Created`; el resultado depende del servidor.	Permitió pasar a otras cuestiones de sintaxis y comunicación técnica.

5	¿Qué significa «Presenta tu diagrama en plenaria»?	Comprender una instrucción académica.	Plenaria, exposición ante toda la clase, explicación del diagrama y preguntas.	Fue una consulta independiente del tema técnico principal.
6	En formato JSON, ¿la coma separa miles y el punto representa los decimales?	Comprender cómo se representan los números en JSON.	Sintaxis numérica, punto decimal, números sin separadores de miles y strings numéricos.	Facilitó el análisis de errores de sintaxis en un cuerpo JSON.
7	¿Qué código devuelve el body de la imagen y por qué?	Identificar errores en un cuerpo JSON y determinar la respuesta HTTP probable.	Comillas dobles, cero inicial en números, separador decimal, booleanos `true` y `false`, `400 Bad Request`.	La conversación avanzó hacia la configuración de un proyecto de Spring Boot.
8	¿Cómo hacer un downgrade a Spring Boot 3.5.16 y Kotlin 2.0.21?	Configurar versiones específicas de las tecnologías del proyecto.	Spring Boot, Kotlin, Maven, `pom.xml`, Java 21 y versiones de complementos.	La modificación de versiones llevó a comprobar la compilación del proyecto.
9	¿Por qué aparece `BUILD FAILURE` con `kotlin-maven-plugin:2.0.21` y `IllegalArgumentException: 27`?	Investigar un error de compilación relacionado con la versión de Java.	Kotlin Maven Plugin, Java 27, compatibilidad, `JAVA_HOME`, JDK 21 y `mvn -version`.	Permitió profundizar en los comandos de Maven y el proceso de compilación.
10	¿Para qué sirve `.\mvnw.cmd compile` y qué significa?	Entender el comando de compilación en Windows.	Maven Wrapper, `mvnw.cmd`, fase `compile`, `pom.xml` y carpeta `target/classes`.	Sirvió de base para comprender las herramientas del proyecto.
11	Explica qué son Maven, Spring y Kotlin, y el flujo de Modelo-Controlador.	Comprender las tecnologías y la organización de una aplicación.	Lenguaje de programación, framework, gestión de dependencias, MVC, modelo, vista, controlador, servicio y repositorio.	Facilitó la comprensión de las peticiones HTTP utilizadas para probar una API.
12	¿Qué significa `curl -i` y para qué sirve?	Aprender a realizar y verificar	`curl`, opción `-i`, encabezados, cuerpo, `-X`	Permitió profundizar en los tipos de

		peticiones HTTP desde la terminal.	`POST`, `-H`, `-d` y códigos HTTP.	contenido aceptados y devueltos por una API.
13	¿Qué significa la cabecera `application/*+json`?	Comprender los tipos de contenido JSON especializados.	Tipos MIME, comodín `*`, sufijo `+json`, `application/problem+json`, `Accept` y `Content-Type`.	Consolidó los conocimientos sobre cabeceras HTTP y formatos de datos.
14	Generar un registro de trazabilidad resumido que identifique las consultas, sus objetivos, la información obtenida y la relación entre ellas, sin inventar interacciones.	Documentar el proceso de investigación y aprendizaje seguido durante la conversación.	Registro cronológico, objetivos, palabras clave, resultados y continuidad entre consultas.	Culmina el proceso mediante la documentación de las consultas realizadas.

Conclusión del proceso

La investigación comenzó con la estructura de las peticiones HTTP y la interpretación de JSON. Después abordó los errores de formato y los códigos de respuesta, continuó con la configuración y compilación de un proyecto Spring Boot con Kotlin y Maven, y finalizó con herramientas para probar APIs y comprender las cabeceras HTTP.

Referencias

Bray, T. (Ed.). (2017). RFC 8259: The JavaScript Object Notation (JSON) data interchange format. IETF. https://www.rfc-editor.org/rfc/rfc8259

Fielding, R., Nottingham, M., & Reschke, J. (Eds.). (2022). RFC 9110: HTTP semantics. IETF. https://www.rfc-editor.org/rfc/rfc9110

Mozilla Developer Network. (s. f.). HTTP messages. https://developer.mozilla.org/en-US/docs/Web/HTTP/Messages

Spring Boot Reference Documentation 3.5. Kotlin Support. https://docs.spring.io/spring-boot/3.5/reference/features/kotlin.html

Guía «Kotlin y Spring Boot 3 con Maven» (material del curso).

Lista de cotejo y puntajes

Instrumentos del PEA para este tema: presentación en plenaria y lista de cotejo. El docente marca la columna «Cumple» con Sí, Parcial o No.

Parte	Criterio	Puntos	Cumple
A	A1. Método, ruta y versión correctos	4	
A	A2. Cabeceras nombradas y explicación de Content-Type	4	
A	A3. Código 201 y su significado; función de Location	4	
A	A4. Los cuatro tipos JSON correctos	4	
A	A5. Predicción justificada	4	
B	Nombra y completa las tres partes de la solicitud	5	
B	Nombra y completa las tres partes de la respuesta	5	
B	Muestra el JSON con sus campos y tipos en los cuerpos	5	
B	Presenta en plenaria y explica el recorrido	5	
C	C1. Identifica y corrige los cuatro errores	12	
C	C2. JSON completo y válido	4	
C	C3. Explica el código 400	4	
D	D1. pom.xml con Spring Boot 3.5.16 y Kotlin 2.0.21	6	
D	D2. Tres data class con los campos y tipos pedidos	8	
D	D3. Controlador con GET, GET por id (404) y POST (201 y Location)	10	
D	D4. Tabla de análisis completa y coherente con las respuestas reales	6	
E	E1. Reflexión que explica el recorrido con los cuatro elementos	6	
E	E2. Declaración de uso de IA completa	4	
	Total	**100**	

Puntaje por parte:

Parte A	Parte B	Parte C	Parte D	Parte E
20	20	20	30	10